AI-Native Retrieval Engine for Agentic Workflows

Project summary from start to current state · FAST-NUCES Karachi, BS(CS) Final Year Project, Fall 2026 – Spring 2027

Team: Kabir Raj · Samay Kumar · Vishal Kothari · Generated 12 Oct 2026 from the repository (branch `k/controlled-path`, commit `8b966ec`)

> **In one paragraph.** We built a standalone retrieval-control middleware service in Python (FastAPI) that sits between a host RAG/agent application and a Qdrant vector database. Instead of retrieving once or looping a fixed number of times, the engine keeps a task state, plans sub-goals ("gaps"), and at every step decides with deterministic rules and a utility function which retrieval action to take, which candidates become evidence, and when to stop, all within an explicit budget. It returns an **EvidenceBundle** (evidence, open/resolved gaps, trace summary, stop reason). The engine never writes the final answer; the host application does.

1. Problem and objective

Retrieval-augmented agents usually do one-shot retrieval or a fixed number of hops. Multi-hop questions need evidence from several documents: one-shot retrieval misses part of it, and fixed loops spend calls whether or not they help. The goal is a service that retrieves *adaptively* under a budget and explains every decision through traces.

- **Controller-OFF (direct path):** one search (+ optional rerank), no state. Used by the baselines and the `/v1/retrieve/direct` endpoint.
- **Controller-ON (controlled path):** a stateful loop over task state S_t: plan → select action → cache/REUSE gate → retrieve → rerank → update evidence → monitor → continue / replan / stop.

2. Timeline: what was built, in order

Date	Work completed
Up to 25 Sep 2026	Proposal defended (Week 6). Full design package written: PRD, architecture, state spec, 20+ module specs, glossary, ADR template.
4 Oct	Baseline import of the design package; package skeleton, pyproject, venv tooling, pre-commit; typed exception hierarchy with API error mapping; `EngineConfig` mirroring `base.yaml` with loader and validation.
5 Oct (foundations)	All shared Pydantic models (S_t, enums, bundle), vector-aware serialization and invariant checks; abstract interfaces for every module and the factory; test fakes, fixtures and a mini corpus; AST import-boundary tests; `requirements.lock` (with Windows Smart App Control workarounds); LangGraph characterised with the nested GraphState. Tag **v0.1-skeleton**.
5 Oct (data & retrieval)	Qdrant v1.19.1 pinned as a native Windows binary; verified MultiHop-RAG download (SHA-256); ingestion: 384-token chunking, UUIDv5 ids, gold map, dev/test split; embedder (bge-base-en-v1.5), BM25 sparse encoder, index writer and Qdrant adapter (dense, sparse, hybrid RRF). Index `multihop_v1` built: 4,441 chunks in 59 min.
5 Oct (engine)	Controller-OFF path `Engine.retrieve_direct` + `/v1/retrieve/direct`; task-local cache with exact/semantic lookup and a cross-task leakage test; latency profile script; JSONL trace writer; eval harness v0 with `dense_oneshot`, metrics, run folders and manifests; `hybrid_oneshot` baseline; SQLite TaskStore with optimistic version check; LangGraph controlled loop with `Engine.step/run`; cross-encoder and state-aware reranker; rule+utility controller, trajectory monitor and evidence manager; LLM client (providers, response cache, per-run caps); bounded LLM planner with validator, fallback and prompts; `/v1/tasks` routes and the Python `RetrievalClient`; `fixed_iteration` and `controller_on` systems in the runner; golden-trajectory, fault-injection and OpenAPI snapshot tests; `show_trace.py`.
8 Oct	Fixed defects found in the first live controller-ON runs; local Ollama models (qwen2.5:3b), reranker score cache, relevance threshold; separate rerank latency on the direct path (ADR-012 A1); answer generation, statistics, reports and premature-stop metrics; measured TRANSFORM latency (14.5 s); kept scored candidates through acceptance and added CER@10; ADR-001/003/004/009/010 accepted and ADR-013 (preliminary calibration) added; FYP-I demo (controller-OFF vs ON over HTTP); aborted runs marked in the manifest.
9 Oct	Distinct run ids for same-second runs; repeated-document rate and evidence-gain-per-retrieval metrics; reportable E0 runs; ADR-005 accepted (bge-reranker-base); hand review of 21 trajectories with 7 failure categories; FYP-I report draft and slides outline with demo runbook; tags **v0.2-week10** and **v0.3-midterm**; integration test proving ingestion reproducible on real files; gap.created / superseded / exhausted events and PRD acceptance tests.
10 Oct	WIP whole-project snapshot shared with partners.

3. Architecture

- **Stack:** Python 3.11, FastAPI, LangGraph (one iteration per graph invoke; `Engine.run` loops), Qdrant 1.19.1 (dense HNSW + sparse BM25 named vectors), SQLite TaskStore as the single source of state, JSONL traces. Windows 10/11 only.
- **Modularity:** each module package has an `interfaces.py` (ABC) plus implementations; concrete classes are chosen only in `engine/factory.py` from config. Every threshold, weight, model name and k value is a config key in `configs/base.yaml`.
- **Models:** `BAAI/bge-base-en-v1.5` embeddings, Qdrant/bm25 sparse, `BAAI/bge-reranker-base` cross-encoder (CPU), `qwen2.5:3b` on local Ollama for planner and answer generator (no API cost).
- **Security:** retrieved text is untrusted, wrapped in delimiters and never placed in the system role; a planner prompt-injection test is in the suite.

Modules

Module	Responsibility
`engine/state`	All shared models, SQLite TaskStore (WAL, version check), serialization, invariants
`engine/retrieval`, `embeddings`	Qdrant adapter (dense / sparse / hybrid RRF), shared embedder, BM25 encoder, index writer
`engine/planner`	LLM decomposes the goal into 1–4 gaps with queries; INIT / REWRITE / SPLIT; schema validation, grounding (no undeclared entities), retries, deterministic fallback
`engine/controller`	Deterministic action policy: preconditions, then U = α·gain − β·cost − γ·redundancy − δ·latency; argmax with fixed tie order; minimum-utility stop per gap
`engine/cache`	Task-local cache keyed by `task_id` (exact + semantic lookup), REUSE action
`engine/rerank`	Cross-encoder scorer with score cache; state-aware reranker
`engine/evidence`	Acceptance on relevance, novelty, duplication; gap resolution by embedding coverage; evidence gain
`engine/monitor`	Trajectory monitor (stagnation, stop checks) and JSONL trace writer
`engine/llm`	Single entry for every LLM call: providers, response cache, usage/cost caps
`engine/graph`, `core.py`	LangGraph nodes; Engine facade: create_task / step / run / result / retrieve_direct, per-task lock, timeouts
`engine/api`, `client/`	FastAPI `/v1/health`, `/v1/retrieve/direct`, `/v1/tasks/*` (create, step, run, result, trace); Python `RetrievalClient`
`baselines/`	`dense_oneshot`, `hybrid_oneshot` (BM25 + dense, RRF), `fixed_iteration` (N = B planner-driven hops)
`eval/`	Runner, metrics, answer generator, statistics (paired bootstrap, exact McNemar), reports, manifests
`data/`, `scripts/`, `examples/`	Download + ingestion; `show_trace.py`, `latency_profile.py`, `check_gold_mapping.py`; FYP-I HTTP demo

Actions: DENSE, SPARSE_HYBRID, RERANK_EXPAND, REUSE (cache), TRANSFORM (planner rewrite/split), STOP.
Stop reasons: all gaps resolved, budget exhausted, stagnation, or low marginal utility on every open gap.

4. Size of the work

Area	Python files	Lines	Area	Python files	Lines
engine/	66	6,644	data/	9	1,096
tests/	96	10,715	scripts/	4	491
eval/	9	1,129	baselines/	6	391
client/	3	270	examples/	3	91

64 commits · 1,019 tests (1,002 unit tests on fakes + integration with live Qdrant, e2e golden trajectories, LLM tests) · ~98 % coverage · `ruff` + `mypy --strict engine` · ~4,900 lines of documentation · 13 ADRs · 24 experiment configs (baselines, controller-ON, 7 ablations, each for dev and test).

5. Requirements status (end of FYP-I)

FR	Item	Status
FR1	Ingestion, chunking, Qdrant index, dense/sparse/hybrid adapter	Done
FR2	Task state S_t, SQLite TaskStore	Done
FR3	Controller-OFF direct path (NFR-P1: median 104–142 ms)	Done
FR4	Planner INIT/REWRITE/SPLIT, validator, fallback	Done
FR5	Task-local cache + REUSE, leakage test	Done
FR6	Utility controller, preconditions, fixed-policy ablation	Done
FR7	State-aware reranking (prototype, weights Draft)	Done
FR8	Evidence manager, gap resolution, trajectory monitor	Done (LLM verifier not built)
FR9	Engine facade, timeouts, per-task lock	Done
FR10	Traces, events, manifests, eval harness, statistics, reports	Done (redundant-evidence rate: FYP-II)
FR11	HTTP API + Python client; FYP-I demo	Done (agent demo: FYP-II)
FR12	MultiHop-RAG loader	Done (HotpotQA, curated set: FYP-II)

6. Data

MultiHop-RAG: 609 news articles, 2,556 queries (ADR-003). 4,441 chunks of 384 tokens; gold mapping 100 % (6,084 / 6,084). Split 1,789 dev / 767 test; dev subset of 300 questions (265 with gold evidence, 35 null queries). The test split has not been touched.

7. Results (dev subset, preliminary)

E0: one-shot baselines (reportable runs, reranker on)

System	R@5	R@10	Hit@10	NDCG@10	MRR	CER@10	CER	Coverage
dense_oneshot	0.459	0.565	0.917	0.487	0.633	0.226	0.328	0.661
hybrid_oneshot	0.497	0.619	0.917	0.522	0.643	0.313	0.445	0.729

Hybrid − dense over 265 paired tasks (95 % bootstrap CI): Recall@10 +0.053 [+0.029, +0.078]; CER@10 +0.087 [+0.049, +0.128], McNemar $p < 0.0001$; CER +0.117 [+0.068, +0.166], $p < 0.0001$. Gains are largest on comparison and inference questions. The reranker adds Recall@10 +0.089 on dense and +0.030 on hybrid.

CER = complete evidence recall (all gold groups present). Runs: `20261008-184814_dense_oneshot_dev_f086ac2`, `20261008-185336_hybrid_oneshot_dev_f086ac2`.

First controller-ON runs (40 tasks, B = 4, not reportable)

CER 0.152, R@10 0.467, trajectory coverage 0.854, 2.65 retrieval calls per task. The full E1 equal-budget run ($B \in \{2, 3, 4\}$) stopped at 56/300 tasks because the laptop ran low on memory, and still has to be re-run.

Latency profile (median)

REUSE 3.8 ms · DENSE 139 ms · SPARSE_HYBRID 154 ms · RERANK_EXPAND 168 ms · TRANSFORM 14.5 s (local 3B LLM on CPU).

8. Trajectory review: what we learned

21 controller-ON trajectories read by hand; counts over 40 tasks. Of 89 gold groups, 43 % reached the final bundle, 42 % were retrieved but not kept, and 16 % were never retrieved. Losing gold at acceptance is the biggest loss.

#	Failure category	FYP-II fix
F1	Relevance scored against short planner subgoal queries, so 30 of 37 lost gold groups fail the relevance threshold	Also score against gap/goal; calibrate threshold
F2	64 % of planner queries name the publisher, which exists only as metadata	Planner guardrail; optional source filter
F3	53 % of gap resolutions made by non-gold passages, causing early stops	Calibrate coverage threshold; verifier band
F4	Second search repeats the same query in another mode with ~0 gain	Controller preference for TRANSFORM
F5	Planner over-splits into near-duplicate gaps	Reject near-duplicate gaps
F6	Planner output rejected (undeclared entity, count, schema); weak REWRITE	Failure analysis; stronger model check
F7	Unanswerable questions spend 2–4 calls before stopping	Early stop when nothing relevant is found

No crashes, invariant violations or REUSE loops occurred. These are calibration and prompt issues, not code defects.

9. Architecture decision records

ADR	Decision	Status
001	Embedding model, chunking, sparse method, collection naming	Accepted
002	Synchronous core; TaskStore as single state source	Accepted
003	Dataset source, licence, checksums, split, answer metric	Accepted
004	LLM models: qwen2.5:3b on local Ollama	Accepted
005	Cross-encoder: bge-reranker-base	Accepted
009	State contract amendments	Accepted
010	API request limits from configuration	Accepted
011	Deviations from the proposal (Windows-only, no LangGraph checkpointer, SQLite-only cache, …)	Awaiting supervisor
012	`DirectResult` return type (+ separate rerank latency)	Accepted
013	Preliminary calibration (relevance threshold 0.2, rerank top-n 20)	Accepted (keys Draft)
006 / 007 / 008	Final calibration freeze, HotpotQA, curated benchmark	FYP-II

10. Open items and limitations

- Not yet done for FYP-I: full E1 run on the dev subset, tag `v0.5-fyp1`, CI run on GitHub, supervisor approval of ADR-011, recorded backup demo video.
- All numbers are from the dev subset, one run per configuration, with preliminary calibration. There are no test-split numbers yet.
- CPU-only reranking and a 3B local LLM make planner calls take 15–50 s, so latency NFRs for the controlled path are not met on the team laptop. Quality numbers are unaffected.

11. FYP-II plan

When	Work	Owner
Jan 2027	Evidence-manager relevance and coverage fixes, stopping for unanswerable questions; planner guardrails; cache/REUSE experiments and source filter	Samay / Vishal / Kabir
Feb 2027	Calibration on the full dev split; ADR-006; freeze config (`v0.8-frozen-config`)	Samay + Vishal
Mar 2027	E1/E2 on the test split (3 repeats), E3 ablations, statistics	Vishal + Samay
Mar–Apr 2027	HotpotQA (E5) and curated benchmark (E6)	Kabir + Vishal / All
Apr–May 2027	LangGraph agent demo / MCP; wheel + Windows deployment docs	Vishal / Kabir
May–Jun 2027	Code freeze, reproducibility package, thesis, viva	All

Sources: `docs/reports/FYP1_REPORT_DRAFT.md`, `docs/reports/FYP1_TRAJECTORY_REVIEW.md`, `docs/TASKS.md`, `docs/decisions/` and the git history. Parts of the code, tests and documentation were written with Claude Code under the team's review.